

Practice 1 – Create process

Tài liệu yêu cầu chạy hello_wait, tìm PID của process đang chạy, rồi truy cập đúng thư mục /proc/<pid>. ProcFS ánh xạ mỗi process vào một thư mục /proc/<pid>, và ps là một cách để tìm process đang chạy.

Công việc cần thực hiện

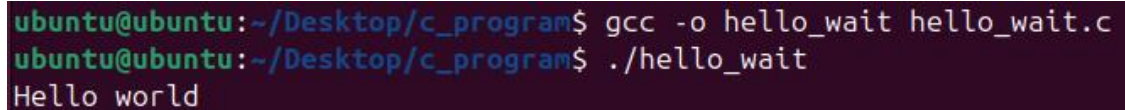
1. Compile và chạy chương trình hello_wait.
2. Khi chương trình đang chờ input, dùng ps aux f | grep hello để tìm process tương ứng.
3. Xác định PID của hello_wait.
4. Dùng PID đó để mở thư mục /proc/<pid>.

Ảnh cần nộp

1. Ảnh chạy ./hello_wait

Trong ảnh cần thấy:

- o lệnh ./hello_wait
- o chương trình đang chạy và chưa kết thúc



```
ubuntu@ubuntu:~/Desktop/c_program$ gcc -o hello_wait hello_wait.c
ubuntu@ubuntu:~/Desktop/c_program$ ./hello_wait
Hello world
```

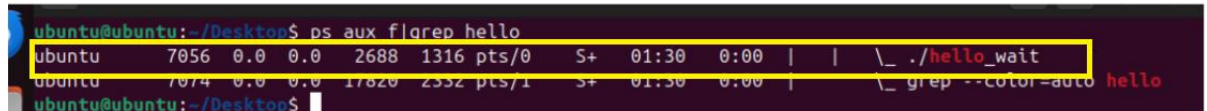
o

2. Ảnh chạy ps aux f | grep hello

Trong ảnh cần thấy:

- o lệnh ps aux f | grep hello
- o dòng process hello_wait

Cần khoanh/đánh dấu: dòng tương ứng với process hello_wait



```
ubuntu@ubuntu:~/Desktop$ ps aux f | grep hello
ubuntu  7056  0.0  0.0  2688  1316 pts/0    S+   01:30   0:00 | | \_ ./hello_wait
ubuntu  7074  0.0  0.0  17620  2352 pts/1    S+   01:30   0:00 | \_ grep --color=auto hello
```

o

3. Ảnh chạy ls /proc/<pid>

Trong ảnh cần thấy:

- o lệnh ls /proc/<pid> với đúng PID đã tìm được

- o nội dung thư mục /proc/<pid>

```
ubuntu@ubuntu:~/Desktop$ ps aux f|grep hello
ubuntu      7056  0.0  0.0   2688  1316 pts/0    S+   01:30   0:00 | | \_ ./hello_wait
ubuntu      7074  0.0  0.0  17820  2332 pts/1    S+   01:30   0:00 | \_ grep --color=auto hello
ubuntu@ubuntu:~/Desktop$ ls /proc/7056
arch_status  cpuset      latency     ns           sched        syscall
attr         cwd         limits      numa_maps    schedstat     task
autogroup    environ    loginuid    oom_adj       sessionid     timens_offsets
auxv         exe         map_files   oom_score     setgroups     timers
cgroup       fd          maps        oom_score_adj smaps          timerslack_ns
clear_refs   fdinfo      mem         pagemap       smaps_rollup  uid_map
cmdline      gid_map     mountinfo   patch_state   stack         wchan
comm         io          mounts      personality   stat
coredump_filter ksm_merging_pages mountstats   projid_map    statm
cpu_resctrl_groups ksm_stat    net         root          status
ubuntu@ubuntu:~/Desktop$
```

o

Câu hỏi cần trả lời

1. PID của process hello_wait là bao nhiêu?

Trả lời: 7056

2. Vì sao sau khi chạy ./hello_wait, chương trình vẫn còn tồn tại và chưa kết thúc?

Trả lời: Chương trình chỉ kết thúc chỉ khi chương trình thực thi đc giải phóng hoàn toàn, nhưng do còn giữ một lệnh getc(stdin) làm cho chương trình vẫn đợi mà chưa kết thúc.

3. Thư mục /proc/<pid> tương ứng với process nào?

Trả lời: Tương ứng với process của hello_wait

Practice 2 – Traverse the tree of processes

Tài liệu yêu cầu chạy hello_fork để tạo thêm process con bằng fork(), sau đó xác định process cha và process con thông qua ps, Pid, PPid, và file children. fork() tạo child process từ parent process; quan hệ cha–con có thể được lần theo bằng PPid và danh sách child trong /proc/<pid>/task/<pid>/children.

Công việc cần thực hiện

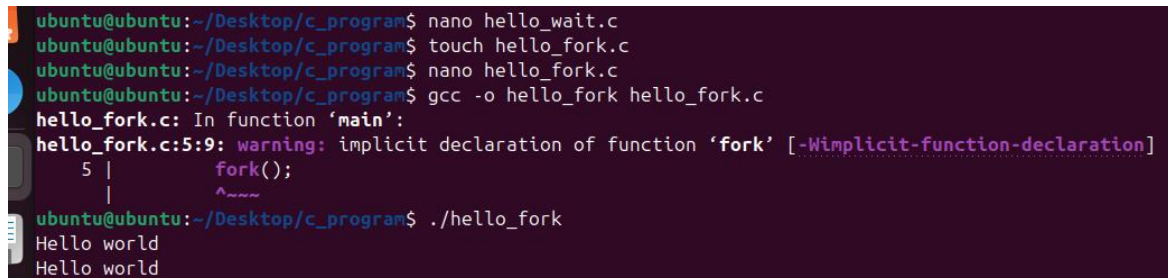
1. Compile và chạy chương trình hello_fork.
2. Quan sát output của chương trình.
3. Dùng ps aux f | grep hello_fork để tìm hai process liên quan.
4. Mở /proc/<pid>/status để xem Pid và PPid.
5. Mở /proc/<pid>/task/<pid>/children để xem child PID từ phía process cha.

Ảnh cần nộp

1. Ảnh chạy ./hello_fork

Trong ảnh cần thấy:

- lệnh ./hello_fork
- output có 2 dòng Hello world



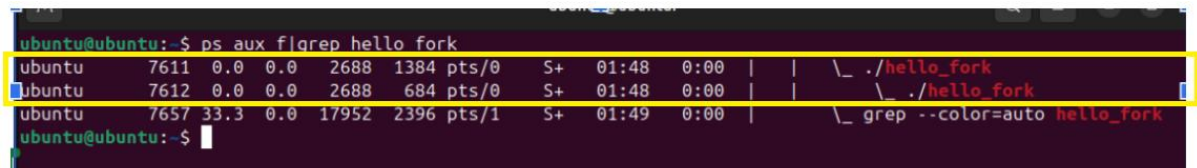
```
ubuntu@ubuntu:~/Desktop/c_program$ nano hello_wait.c
ubuntu@ubuntu:~/Desktop/c_program$ touch hello_fork.c
ubuntu@ubuntu:~/Desktop/c_program$ nano hello_fork.c
ubuntu@ubuntu:~/Desktop/c_program$ gcc -o hello_fork hello_fork.c
hello_fork.c: In function 'main':
hello_fork.c:5:9: warning: implicit declaration of function 'fork' [-Wimplicit-function-declaration]
     5 |         fork();
       |         ^~~~~
ubuntu@ubuntu:~/Desktop/c_program$ ./hello_fork
Hello world
Hello world
```

2. Ảnh chạy ps aux f | grep hello_fork

Trong ảnh cần thấy:

- lệnh ps aux f | grep hello_fork
- kết quả có 2 process hello_fork

Cần khoanh/đánh dấu: 2 dòng process hello_fork



```
ubuntu@ubuntu:~$ ps aux f | grep hello_fork
ubuntu    7611  0.0  0.0  2688  1384 pts/0    S+   01:48   0:00 | | \_ ./hello_fork
ubuntu    7612  0.0  0.0  2688    684 pts/0    S+   01:48   0:00 | | \_ \_ ./hello_fork
ubuntu    7657 33.3  0.0 17952  2396 pts/1    S+   01:49   0:00 | \_ grep --color=auto hello_fork
ubuntu@ubuntu:~$
```

3. Ảnh chạy head -n 10 /proc/<pid>/status

Trong ảnh cần thấy:

- lệnh head -n 10 /proc/<pid>/status

- các trường Pid và PPid

Cần khoanh/đánh dấu: Pid và PPid

- **PPid (7611)**

```

need: invltd number of lines: 10/proc/7611/status
ubuntu@ubuntu:~$ head -n 10 /proc/7611/status
Name:   hello_fork
Umask:  0002
State:  S (sleeping)
Tgid:   7611
Ngid:   0
Pid:    7611
PPid:   3856
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
ubuntu@ubuntu:~$

```

○

○ Pid (7612)

```

ubuntu@ubuntu:~$ head -n 10 /proc/7612/status
Name:   hello_fork
Umask:  0002
State:  S (sleeping)
Tgid:   7612
Ngid:   0
Pid:    7612
PPid:   7611
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
ubuntu@ubuntu:~$

```

4.

5. Ảnh chạy `cat /proc/<pid>/task/<pid>/children`

Trong ảnh cần thấy:

- lệnh `cat /proc/<pid>/task/<pid>/children`
- child PID của process cha

Chỉ cần khoanh/đánh dấu nếu ảnh có nhiều dòng hoặc khó nhìn.

```

ubuntu@ubuntu:~$ cat /proc/7611/task/7611/children
7612  ubuntu@ubuntu:~$
HELLO WORLD

```

Câu hỏi cần trả lời

1. Vì sao chương trình `hello_fork` in ra 2 dòng Hello world?

Trả lời: Bởi vì lệnh `fork()` ban đầu được gọi đc xem như là một lệnh gọi quá trình mới bằng cách nhân đôi lời gọi hàm ban đầu. Quá trình mới này được xem là child process của parent process. Do đó, lệnh gọi ban đầu in ra “Hello world” do có lệnh `fork()` nên lệnh này sẽ được gọi lại thêm lần nữa.

2. Trong 2 process tìm được, process nào là parent, process nào là child?

Trả lời : Process 7611 là process cha ; còn process 7612 là process con.

3. PPid của process con là gì, và nó cho biết điều gì?

Trả lời : PPid của process con là mã định danh của tiến trình cha đã trực tiếp tạo ra nó và nó cho biết tiến trình cha đã tạo ra tiến trình con đang xét đến.

4. File children của process cha cho thấy thông tin gì?

Trả lời: Danh sách PID các process con mà ở process cha của nó đang trực tiếp chiếm hữu.

Practice 3 – Examine the process memory regions

Tài liệu yêu cầu chạy multivar để in ra địa chỉ của các đối tượng trong process, sau đó mở /proc/<pid>/maps để đối chiếu với các vùng nhớ. Memory layout cơ bản của process gồm **text**, **data**, **heap**, **stack**; file /proc/<pid>/maps hiển thị các memory regions đã được ánh xạ và có các mục như [heap] và [stack].

Công việc cần thực hiện

1. Compile và chạy chương trình multivar.
2. Ghi nhận địa chỉ của:
 - glo_init_data
 - glo_noninit_data
 - func
 - local_data
3. Dùng ps aux f | grep multivar để lấy PID của process.
4. Mở /proc/<pid>/maps.
5. Đối chiếu các địa chỉ in ra từ chương trình với các vùng nhớ trong maps.

Ảnh cần nộp

1. Ảnh chạy ./multivar
Trong ảnh cần thấy:
 - lệnh ./multivar
 - Process ID
 - địa chỉ của glo_init_data
 - địa chỉ của glo_noninit_data
 - địa chỉ của func
 - địa chỉ của local_data

```
ubuntu@ubuntu:~/Desktop/c_program$ ./multivar
Process ID= 8222
Addresses of the process :
1. glo_init_data = 1016582160
2. glo_noninit_data = 0x592f3c97d018
3. print_func ( ) = 0x592f3c97a1c9
4. local_data = 0x7ffe5668f170
```

2. Ảnh chạy ps aux f | grep multivar

Trong ảnh cần thấy:

- lệnh ps aux f | grep multivar
- dòng process multivar

Cần khoanh/đánh dấu: dòng đúng của process multivar

```
1 ubuntu@ubuntu:~/Desktop/c_program$ ps aux | grep multivar
ubuntu      8169  7.0  0.0  2688  1612 pts/0    S+   02:21   0:24 | | \_ ./multivar
ubuntu      8222  6.0  0.0  2688  1600 pts/1    S+   02:24   0:12 | | \_ ./multivar
ubuntu      8291  7.6  0.0  17952  2456 pts/2    S+   02:27   0:00 | | \_ grep --color=auto multivar
ubuntu@ubuntu:~/Desktop/c_program$
```

○

3. Ảnh chạy cat /proc/<pid>/maps

Trong ảnh cần thấy:

- lệnh cat /proc/<pid>/maps
- vùng mapping của executable
- [heap]
- [stack]

Cần khoanh/đánh dấu:

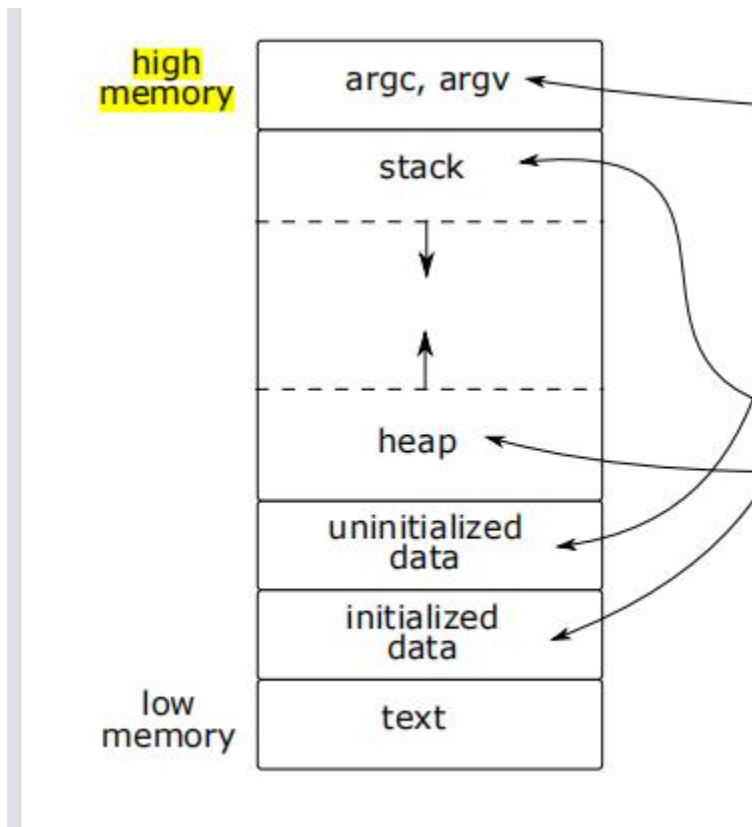
- vùng executable của chương trình
- [heap]
- [stack]

```

64c9b8bb2000-64c9b8bb3000 r--p 00000000 00:1c 23123 /home/ubuntu/Desktop/c_program/multivar (delete
d)
64c9b8bb3000-64c9b8bb4000 r-xp 00001000 00:1c 23123 executable /home/ubuntu/Desktop/c_program/multivar (delete
d)
64c9b8bb4000-64c9b8bb5000 r--p 00002000 00:1c 23123 /home/ubuntu/Desktop/c_program/multivar (delete
d)
64c9b8bb5000-64c9b8bb6000 r--p 00002000 00:1c 23123 /home/ubuntu/Desktop/c_program/multivar (delete
d)
64c9b8bb6000-64c9b8bb7000 rw-p 00003000 00:1c 23123 /home/ubuntu/Desktop/c_program/multivar (delete
d)
64c9d0afd000-64c9d0b1e000 rw-p 00000000 00:00 0 [heap]
7932c084c000-7932c084f000 rw-p 00000000 00:00 0
7932c084f000-7932c0871000 r--p 00000000 00:1c 14975 /usr/lib/x86_64-linux-gnu/libc.so.6
7932c0871000-7932c09ff000 r-xp 00028000 00:1c 14975 /usr/lib/x86_64-linux-gnu/libc.so.6
7932c09ff000-7932c0aa4e000 r--p 001b0000 00:1c 14975 /usr/lib/x86_64-linux-gnu/libc.so.6
7932c0aa4e000-7932c0a52000 r--p 001fe000 00:1c 14975 /usr/lib/x86_64-linux-gnu/libc.so.6
7932c0a52000-7932c0a54000 rw-p 00202000 00:1c 14975 /usr/lib/x86_64-linux-gnu/libc.so.6
7932c0a54000-7932c0a61000 rw-p 00000000 00:00 0
7932c0a61000-7932c0a73000 rw-p 00000000 00:00 0
7932c0a73000-7932c0a77000 r--p 00000000 00:00 0
7932c0a77000-7932c0a79000 r--p 00000000 00:00 0 [vvar]
7932c0a79000-7932c0a7b000 r-xp 00000000 00:00 0 [vvar_vclock]
7932c0a7b000-7932c0a7c000 r--p 00000000 00:00 0 [vdso]
7932c0a7c000-7932c0aa7c000 r--p 00000000 00:1c 14630 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7932c0aa7c000-7932c0aa7000 r-xp 00001000 00:1c 14630 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7932c0aa7000-7932c0ab1000 r--p 00002c000 00:1c 14630 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7932c0ab1000-7932c0ab3000 r--p 000036000 00:1c 14630 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7932c0ab3000-7932c0ab5000 rw-p 000038000 00:1c 14630 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7932c0ab5000-7932c0ab5000 rw-p 00000000 00:00 0
7ffe1c944000-7ffe1c965000 rw-p 00000000 00:00 0 [stack]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0 [vsyscall]

```

Câu hỏi cần trả lời



1. `global_init_data` thuộc vùng nhớ nào?

Trả lời: Thuộc vùng nhớ initialized data.

2. `glob_noninit_data` thuộc vùng nhớ nào?

Trả lời: Thuộc vùng nhớ `initialized data`.

3. `func` thuộc vùng nhớ nào?

Trả lời: Thuộc vùng nhớ `text`.

4. `local_data` thuộc vùng nhớ nào?

Trả lời: Theo mình `local_data` thuộc vùng nhớ `stack` .

5. Dựa trên maps, xác định vùng executable, heap và stack bằng cách nào?

**Trả lời: Trước hết, ta sẽ xác định cấu trúc của một dòng path sẽ được xuất ra như sau :
Địa chỉ bắt đầu - Địa chỉ kết thúc | Quyền hạn | Offset | Device | Inode | Tên vùng (path).
Vì vậy, cách đơn giản để xác định các vùng này chính là đọc trên vùng path với cú pháp [`<Tên vùng>`]. Còn riêng đối với vùng nhớ executable thường đi kèm với các thư mục thực thi để thực hiện**

Practice 4 – Inter Process Communication

Lab yêu cầu thực hiện hai mô hình IPC cơ bản là **shared memory** và **message passing**.

Practice 4.1 – Shared Memory

Tài liệu yêu cầu chạy reader và writer ở hai terminal khác nhau để minh họa shared memory. Phần kết quả và aftermath yêu cầu chú ý hiện tượng reader có thể in ra giá trị rỗng, giải thích theo thứ tự thực thi, đảo thứ tự chạy reader/writer để kiểm tra lại, và có thể đổi message content để verify thêm.

Công việc cần thực hiện

1. Compile reader.c và writer.c.
2. Mở hai terminal riêng biệt.
3. Chạy reader ở một terminal và writer ở terminal còn lại.
4. Quan sát:
 - shared memory ID
 - shared memory mm
 - shared memory content
5. Đảo thứ tự chạy hai chương trình và quan sát lại kết quả.
6. Có thể đổi nội dung message để kiểm tra lại cơ chế chia sẻ dữ liệu.

Ảnh cần nộp

1. Ảnh terminal chạy ./reader

Trong ảnh cần thấy:

- lệnh ./reader
- shared memory ID
- shared memory mm
- shared memory content

```
ubuntu@ubuntu:~/Desktop/c_program$ ./reader
shared memory ID: 32779
shared memory mm: 0x7aaecf021000
shared memory content:
```

2. Ảnh terminal chạy ./writer

Trong ảnh cần thấy:

- lệnh ./writer
- Shared-memory ID
- shared memory mm
- shared memory content : hello world

```
ubuntu@ubuntu:~/Desktop/c_program$ gcc -o writer writer.c
ubuntu@ubuntu:~/Desktop/c_program$ ./writer
Shared-memory ID: 32779
shared memory mm: 0x7239fbf6f000
shared memory content: hello world
```

3. Ảnh hoặc trang đặt kết quả reader và writer cạnh nhau

Trong ảnh/trang cần thấy:

- kết quả của cả hai phía để so sánh

Cần khoanh/đánh dấu:

- shared memory mm của reader
- shared memory mm của writer
- dòng shared memory content

```
ubuntu@ubuntu: ~/Desktop$ cd c_program/
ubuntu@ubuntu:~/Desktop/c_program$ gcc -o writer writer.c
ubuntu@ubuntu:~/Desktop/c_program$ ./writer
Shared-memory ID: 32790
shared memory mm: 0x768b6df85000
shared memory content: hello world

ubuntu@ubuntu:~/Desktop/c_program$

ubuntu@ubuntu:~/Desktop$ cd c_program/
ubuntu@ubuntu:~/Desktop/c_program$ gcc -o reader reader.c
reader.c: In function 'main':
reader.c:31:9: warning: implicit declaration of function 'sleep' [-Wimplicit-function-declaration]
   31 |         sleep(10);
      |         ^~~~~~
ubuntu@ubuntu:~/Desktop/c_program$ ./reader
shared memory ID: 32790
shared memory mm: 0x70e4cda02000
shared memory content: hello world

ubuntu@ubuntu:~/Desktop/c_program$
```

Câu hỏi cần trả lời

1. Hai process reader và writer có cùng PID hay không?

Trả lời:

```
ubuntu@ubuntu:~/Desktop/c_program$ ps aux f | grep --color=auto writer
rep writer
ubuntu 9411 25.0 0.0 17820 2256 pts/1 S+ 03:33 0:00 \_ grep --color=auto writer
ubuntu@ubuntu:~/Desktop/c_program$

ubuntu@ubuntu:~/Desktop/c_program$ ps aux f | grep --color=auto reader
ubuntu 9417 0.0 0.0 17820 2312 pts/0 S+ 03:33 0:00 \_ grep --color=auto reader
ubuntu@ubuntu:~/Desktop/c_program$
```

Rõ ràng là không cùng PID.

2. Giá trị shared memory mm của hai process có giống nhau không?

Trả lời : Dựa trên kết quả ảnh trên ta thấy rõ địa chỉ của 2 vùng process shared memory là khác nhau do khác vị trí địa chỉ trỏ đến trên ô nhớ.

3. Nếu shared memory mm khác nhau, vì sao hai process vẫn đọc/ghi được cùng một nội dung?

Trả lời: Theo em được biết thì shared memory là cách thức giữa hai tiến trình chia sẻ tài nguyên cho nhau bằng cách process A copy memory vào vị trí shared memory chung, process B sẽ ở một vị trí memory khác copy memory của A từ vị trí dùng chung đó, tức là địa chỉ của vùng ô nhớ trỏ đến có thể khác vì chỉ copy nội dung của các tiến trình cho nhau thôi.

4. Khi chạy reader trước writer, quan sát được gì?

```
ubuntu@ubuntu:~/Desktop/c_program$ ./reader
shared memory ID: 32779
shared memory mm: 0x7aaecf021000
shared memory content:
█
```

```
ubuntu@ubuntu:~/Desktop/c_program$ gcc -o writer writer.c
ubuntu@ubuntu:~/Desktop/c_program$ ./writer
Shared-memory ID: 32779
shared memory mm: 0x7239fbf6f000
shared memory content: hello world
```

Trả lời : shared memory content của reader chưa được display.

5. Khi đổi thứ tự chạy, kết quả thay đổi như thế nào?

```
ubuntu@ubuntu:~/Desktop/c_program$ gcc -o writer writer.c
ubuntu@ubuntu:~/Desktop/c_program$ ./writer
Shared-memory ID: 32796
shared memory mm: 0x7e7e96c72000
shared memory content: hello world

ubuntu@ubuntu:~/Desktop/c_program$ gcc -o reader reader.c
reader.c: In function 'main':
reader.c:31:9: warning: implicit declaration of function 'sleep' [-Wimplicit-function-declaration]
   31 |         sleep(10);
      |         ^~~~~~
ubuntu@ubuntu:~/Desktop/c_program$ ./reader
shared memory ID: 32797
shared memory mm: 0x720a2f1c4000
shared memory content:
█
```

- 6.
7. Nếu đổi nội dung message từ hello world sang một nội dung khác, kết quả có còn phù hợp với cơ chế shared memory không?

Trả lời: Theo em, cơ chế đó sẽ không thay đổi. Bởi bản chất của shared memory nó không phụ thuộc vào nội dung mà nó message cho nhau nó chỉ phụ thuộc vào vùng nhớ đó có dùng chung hay không?

Practice 4.2 – Message Queue

Tài liệu yêu cầu chạy msgsnd và msgrcv ở hai terminal khác nhau để minh họa message queue. Kết quả mong đợi là message được gửi từ msgsnd sang msgrcv, và phần aftermath kết luận đây là giao tiếp **một chiều**, tức **half-duplex**.

Công việc cần thực hiện

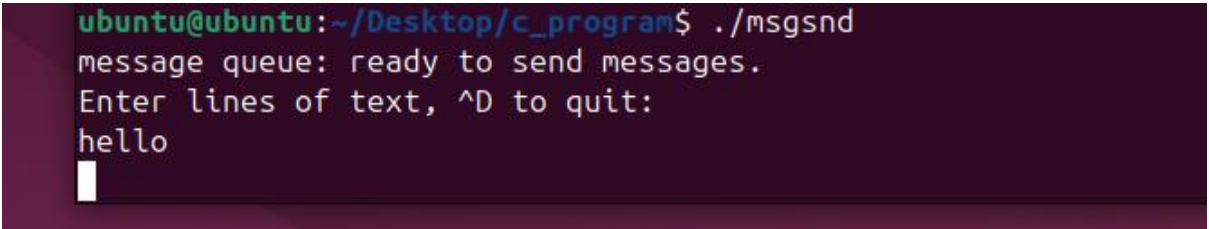
1. Compile msgsnd.c và msgrcv.c.
2. Mở hai terminal riêng biệt.
3. Chạy msgsnd ở một terminal và msgrcv ở terminal còn lại.
4. Nhập ít nhất một message ở phía gửi.
5. Quan sát message ở phía nhận.

Ảnh cần nộp

1. Ảnh terminal chạy ./msgsnd

Trong ảnh cần thấy:

- dòng ready to send message
- ít nhất một message đã nhập

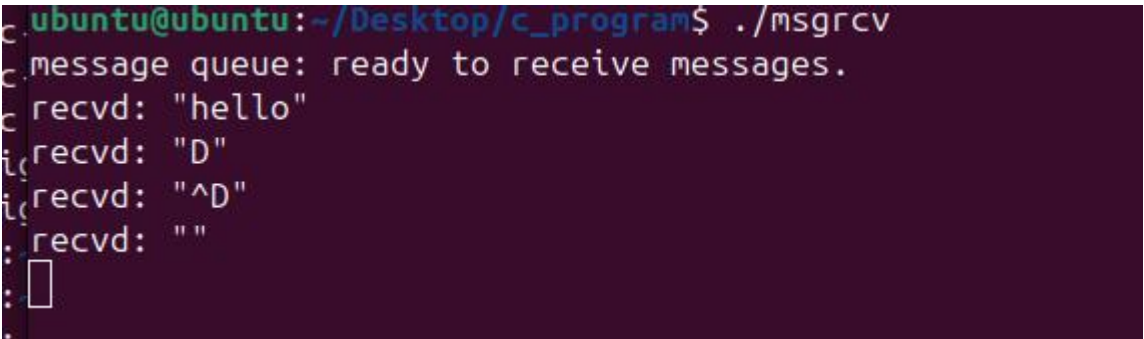
A terminal window with a dark purple background. The prompt is 'ubuntu@ubuntu:~/Desktop/c_program\$'. The command './msgsnd' has been executed. The output shows 'message queue: ready to send messages.' followed by 'Enter lines of text, ^D to quit:'. The word 'hello' has been entered, and a white cursor is visible on the line below it.

```
ubuntu@ubuntu:~/Desktop/c_program$ ./msgsnd
message queue: ready to send messages.
Enter lines of text, ^D to quit:
hello
```

2. Ảnh terminal chạy ./msgrcv

Trong ảnh cần thấy:

- dòng ready to receive message
- message đã nhận được

A terminal window with a dark purple background. The prompt is 'ubuntu@ubuntu:~/Desktop/c_program\$'. The command './msgrcv' has been executed. The output shows 'message queue: ready to receive messages.' followed by several 'rcvd: ' lines. The first line is 'rcvd: "hello"', followed by 'rcvd: "D"', 'rcvd: "^D"', and 'rcvd: " "'. A white cursor is visible on the line below the last 'rcvd' line.

```
ubuntu@ubuntu:~/Desktop/c_program$ ./msgrcv
message queue: ready to receive messages.
rcvd: "hello"
rcvd: "D"
rcvd: "^D"
rcvd: " "
```

3. Nếu gửi nhiều message, **khoanh/đánh dấu** một message cụ thể ở bên gửi và đúng message tương ứng ở bên nhận.

Câu hỏi cần trả lời

1. Message được truyền theo chiều nào?

TL: Message được truyền **từ tiến trình gửi (msgsnd.c) sang tiến trình nhận (msgrcv.c)**.

2. Vì sao bài này được xem là giao tiếp một chiều?

TL: Vì quá trình này chỉ có một bên gửi và một bên nhận ko có quá trình ngược lại

3. “Half-duplex” trong thí nghiệm này nên hiểu như thế nào?

TL: Trong thí nghiệm này, half-duplex nên hiểu là cơ chế giao tiếp có thể hỗ trợ truyền hai chiều, nhưng không đồng thời cùng lúc.

4. Message queue khác shared memory ở điểm nào về cách truyền dữ liệu?

Trả lời : Message queue truyền dữ liệu bằng cách gửi / nhận message qua kernel; Trong khi: Shared memory truyền dữ liệu bằng cách cho các process cùng truy cập chung một vùng nhớ

Practice 5 – Create thread using Pthread library

Tài liệu yêu cầu dùng Pthread để tạo 2 thread trong cùng một process, compile với tùy chọn -pthread, rồi dùng ps để xác nhận rằng chương trình chỉ có **một process** nhưng có nhiều luồng thực thi. Kết quả mong đợi là chỉ có một process hello_thread, có hai execution instances, và xuất hiện 3 thông báo in ra.

Công việc cần thực hiện

1. Compile chương trình hello_thread với tùy chọn -pthread.
2. Chạy chương trình.
3. Quan sát output của chương trình.
4. Dùng ps aux f | grep hello_thread để kiểm tra process tương ứng.

Ảnh cần nộp

1. Ảnh lệnh compile

Trong ảnh cần thấy:

- o lệnh gcc -pthread -o hello_thread hello_thread.c

2. Ảnh chạy ./hello_thread

Trong ảnh cần thấy:

- Hello world
- Thread 1 ...
- Thread 2 ...

```
ubuntu@ubuntu:~/Desktop/c_program$ touch hello_thread.c
ubuntu@ubuntu:~/Desktop/c_program$ nano hello_thread.c
ubuntu@ubuntu:~/Desktop/c_program$ gcc -pthread -o hello_thread hello_thread.c
ubuntu@ubuntu:~/Desktop/c_program$ ./hello_thread
Hello world
Thread 1: holding 10000
Thread 2: holding 20000

```

3. Ảnh chạy ps aux f | grep hello_thread

Trong ảnh cần thấy:

- dòng process hello_thread

Cần khoanh/đánh dấu: dòng process hello_thread

```
ubuntu@ubuntu:~/Desktop/c_program$ ps aux f | grep hello_thread
ubuntu      8633  0.0  0.0  84616  1652 pts/1    Sl+   04:25
0:00 |      \_  ./hello_thread
ubuntu      8677  0.0  0.0  17952  2384 pts/0    S+    04:26
0:00 |      \_  grep --color=auto hello_thread
ubuntu@ubuntu:~/Desktop/c_program$
```

Câu hỏi cần trả lời

1. Vì sao chương trình có 3 thông báo output nhưng chỉ có 1 process hello_thread?

Trả lời : Vì bản chất của Thread (Luồng) là các đơn vị thực thi chạy bên trong cùng một Process (Tiến trình). Một tiến trình khi khởi chạy luôn có sẵn một luồng chính (Main thread). Trong code, bạn tạo thêm 2 luồng mới (thread1 và thread2). Tổng cộng có 3 luồng cùng chạy song song và thực hiện lệnh printf, do đó bạn thấy 3 dòng output. Tuy nhiên, tất cả chúng đều dùng chung không gian địa chỉ, mã nguồn và tài nguyên của duy nhất một tiến trình hello_thread mà hệ điều hành đã cấp phát.

2. Hai thread trong bài được tạo ra bằng lệnh nào?

Trả lời : pthread_create(&thread1, NULL, &f_count, "1"); và pthread_create(&thread2, NULL, &f_count, "2");

3. Tùy chọn -pthread có vai trò gì khi compile?

Trả lời : Tùy chọn -pthread có hai vai trò chính khi biên dịch bằng gcc: Linker: Nó thông báo cho trình liên kết (linker) thêm thư viện libpthread vào chương trình (tương đương với -lpthread). và Compiler: Nó định nghĩa các macro cần thiết (như _REENTRANT) để trình biên dịch biết rằng code này sẽ chạy đa luồng. Điều này giúp các hàm thư viện chuẩn (như printf) hoạt động an toàn trong môi trường đa luồng (thread-safe).

4. Bài này giúp phân biệt process và thread như thế nào?

Trả lời :

Về không gian bộ nhớ (Memory Space): Các thread trong bài cùng truy cập và thay đổi biến toàn cục count. Điều này chứng minh các thread dùng chung vùng dữ liệu (Data Segment) của process. Nếu là 2 process khác nhau, biến count của mỗi bên sẽ độc lập.

Về tính độc lập: Mặc dù dùng chung bộ nhớ, nhưng mỗi thread lại có vòng lặp for với biến i riêng. Điều này cho thấy mỗi thread có Stack riêng và Program Counter riêng để quản lý việc thực thi của chính nó.

Về tài nguyên: Tạo thread nhẹ hơn tạo process rất nhiều vì OS không cần tạo lại bảng trang (page table) hay cấp phát vùng nhớ mới hoàn toàn, mà chỉ cần tạo một ngữ cảnh thực thi (context) mới bên trong process hiện tại.